PNI ソフトウェア健全性診断レポート

対象プロジェクト `legacy-order-hub` (`v0.1.0`)（`examples/python/api_legacy_messy`）に対する PNI (Project Normalized Inspection) ステートフル健全性診断レポートです。3観点のアーキテクチャ構造記録・静的 Python AST 解析・Docker 隔離環境での動的ビルド／オフライン単体テスト／カバレッジ／起動スモーク検証・Trivy 脆弱性スキャン・隔離ループバック動的 HTTP ペネトレーション検証（`web_pentest`）を統合し、7軸で定量評価を実施しました。

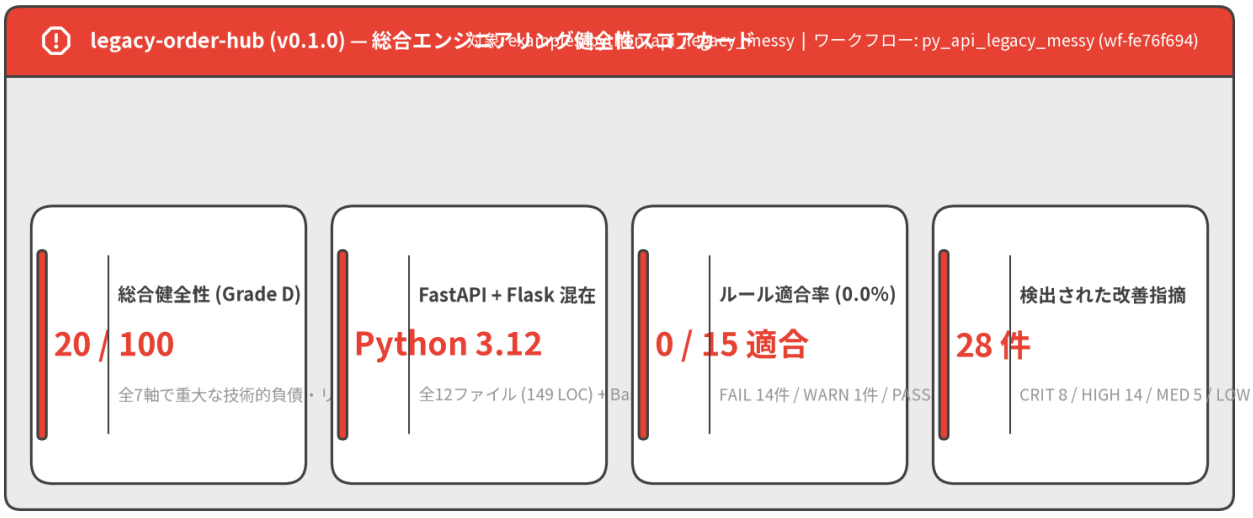


図0-1: legacy-order-hub 総合健全性スコアカード (20/100・Grade D)

診断メタデータ概要

項目	診断記録内容
対象プロジェクト / バージョン	`legacy-order-hub` (`0.1.0` / `setup.py`) ー 顧客注文バッチ処理 HTTP サービスおよび CSV 同期スクリプト
対象リポジトリ相対パス	`examples/python/api_legacy_messy` （ `order_app/**` 12ファイル、 `tests/**` 1ファイル、 `scripts/legacy_sync.sh` ）
主要技術スタック	Python、FastAPI、Flask、Pydantic、Requests、PyYAML、urllib3、Uvicorn、Bash、 `python:latest`
実行ワークフロー / Run ID	`py_api_legacy_messy` / `wf-fe76f694`（全18ステップ完遂・状態DB記録済）
診断実行日時 (UTC)	`2026-10-24T15:06:19Z`
コンテナ動的検証結果	`failed` ー ビルド成功（ `1.00s` 、ハーネス `0.96s` ）、単体テスト `1/1` 成功（ `2.13s` ）、実測行カバレッジ `6.6%` 、起動スモーク `PydanticUserError` **で異常終了（** `1.92s` **）**
セキュリティ & DAST 検査結果	`failed` / `error` ー Python CVE `7件` **（HIGH 4件）**、Debian 13.7 OS CVE `3,554件` **（CRIT 24 / HIGH 436）**、CWE シンク `2件` 、動的 `web_pentest` **起動失敗（** `server_booted=false` , `3.52s` **）**
総合評価 (Overall Grade)	**20 / 100 (Grade D)** ー 起動不能障害・RCE/コマンド注入シンク・特権 CI・肥大 `python:latest` 基盤・DAST ゲート遮断の即時是正が必須

Table of Contents

1. エグゼクティブサマリー

2. アーキテクチャとソースコード構成

3. 推奨する変更対応

4. 検査指標・ルール適合マトリクスと指摘一覧

5. 詳細分析：コード品質・テスト・動的検証

6. 詳細分析：セキュリティ・脆弱性・動的DAST・API境界

1. エグゼクティブサマリー

`legacy-order-hub`（`examples/python/api_legacy_messy`）の総合健全性スコアは **20 / 100 (Grade D)** であり、評価対象15ルール中 **適合** `0件`（`FAIL` **14件**・`WARN` **1件**） という危機的状態にあります。最大の課題は、未固定の `requirements.txt` により導入される Pydantic v2 環境下で `order_app/schemas.py` の未注釈属性 `legacy_flag = True # type: ignore` が `PydanticUserError` **を引き起こし、起動スモーク検証（**`run_check`**）のみならず実コンテナ HTTP 動的ペネトレーション検証（**`web_pentest`**:** `status="error"`, `server_booted=false`, `3.52s`**）でも ASGI アプリが完全起動不能** になっている点です。これにより `.pni/pentests/examples/python/api_legacy_messy/` に整備された DAST プローブ群（`POST /orders` の `CWE-306` 無認証検証・`CWE-20` 未型付け `items` 500 検証等）のライブ実行も遮断（`0/0` 実行）されています。さらに唯一存在するアサーション無しの単体テスト（実測行カバレッジ `6.6%`・分岐 `0.0%`）がこの致命的起動障害を完全に看過しており、`order_app/services/processor.py` には **CWE-502（**`yaml.load`**）** と **CWE-78（**`subprocess.run(..., shell=True)`**）**、ハードコード秘密鍵、可変デフォルト引数バグが集中し、`FROM python:latest`（root 実行）により **計 3,561 件の CVE** を抱えています。

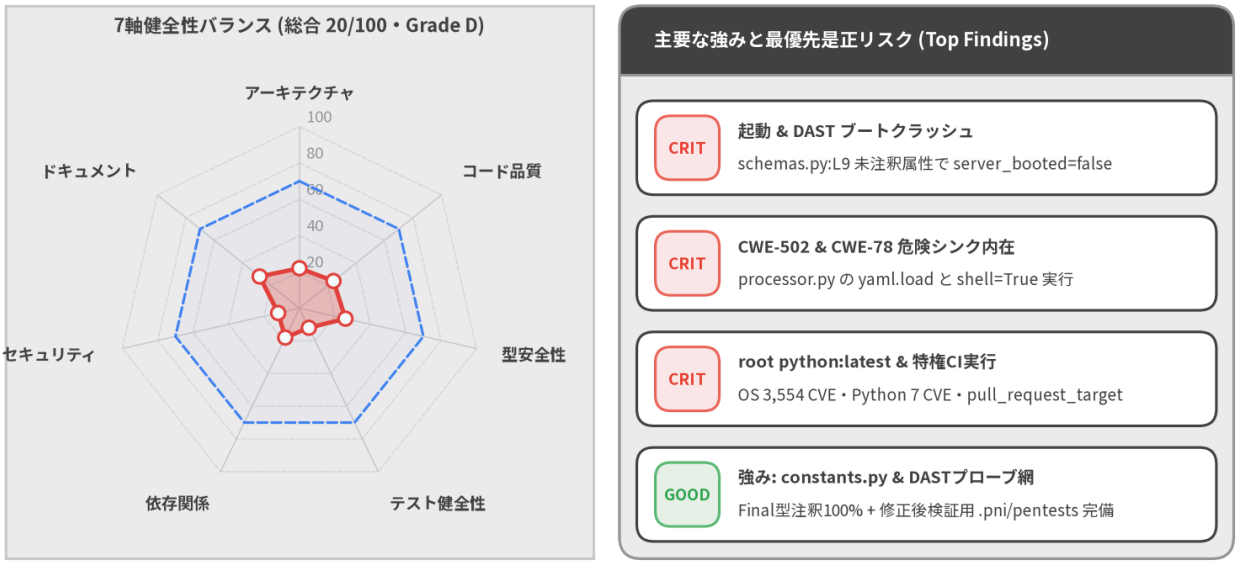


図1-1: 7軸エンジニアリング健全性レーダーチャート（左）と強み・最優先リスク対比（右）

動的検証・セキュリティ・DAST 主要ヘッドライン指標

コンテナ隔離検証（`run_check`）、Trivy スキャン（`security_scan`）、および隔離ループバック HTTP ペネトレーション検証（`web_pentest`）によって、「実運用コンテナの即時インポートクラッシュによる HTTP 完全停止（`server_booted=false`）」と「フルサイズ OS イメージ由来の膨大な脆弱性露出」が動的に裏付けられました。

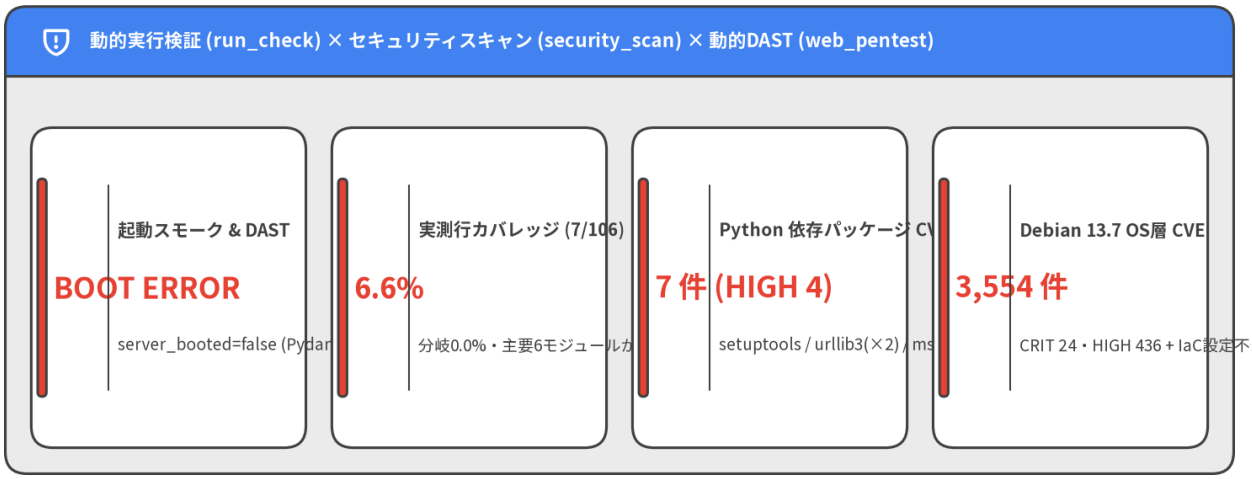


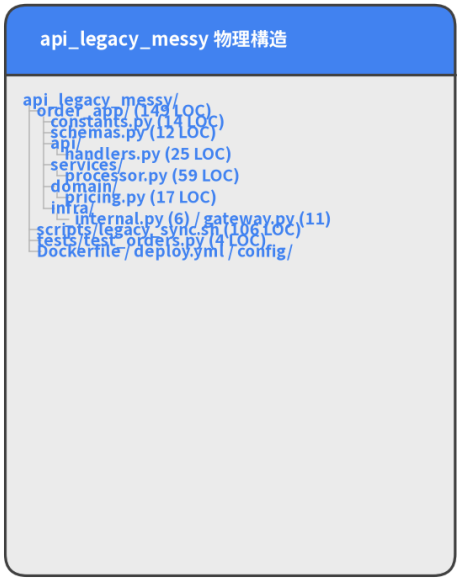
図1-2: コンテナ動的検証・Trivyスキャン・動的DAST (web_pentest) 主要ヘッドラインKPI

7軸評価サマリー

評価軸	スコア	評価根拠の要点
ドキュメント (`documentation`)	**28**	`README.md` は2行のスタブのみ。Docstring 網羅率 `40.9%`（欠落12件、`TODO` 1件、`pricing.py` 引数不一致1件）。
型安全性 (`typing`)	**26**	型注釈率 `44.4%`、`process_order_batch` 全9箇所未注釈、`schemas.py:L9` の `# type: ignore` が起動クラッシュを誘発。
コード品質 (`code_quality`)	**24**	認知的複雑度 `44`（8引数）、可変デフォルト `tags=[]`、裸の `except:`、`async def` 内の同期 `sleep` / `requests.post`。
アーキテクチャ (`architecture`)	**22**	3モジュール循環依存（`handlers -> processor -> pricing -> handlers`）、非公開 `infra._internal` 侵食、FastAPI/Flask 混在。
依存関係 (`dependencies`)	**18**	`requirements.txt` 全8件ピン無し・ロック不在、本番に `flake8` / `black` 混入、`uvicorn` と `boto3` の宣言漏れ。
テスト健全性 (`testing`)	**12**	12ソースに対し `assert` 無しの2行テスト1件のみ。実測行カバレッジ `6.6%`・分岐 `0.0%` で起動不能障害を検知不能。
セキュリティ (`security`)	**12**	Python HIGH CVE 4件、OS CRITICAL 24件、root 実行、CWE-502/78 シンク、秘密鍵直書き、`pull_request_target`、`web_pentest` 起動不能（DAST `CWE-306` / `CWE-20` 未通過）。

2. アーキテクチャとソースコード構成

`legacy-order-hub` はディレクトリ上こそ `api/`・`services/`・`domain/`・`infra/` の4層パッケージ構成を採っていますが、実態は **循環依存・レイヤ逆転・2種類の Web フレームワーク同居・サイドバンド Bash スクリプトが絡み合う劣化した階層型サービス**（Python 12ファイル計 `149 LOC` + `scripts/legacy_sync.sh` `106 LOC`）です。とりわけ `order_app/domain/pricing.py` が上位 `order_app/api/handlers.py` の `ACTIVE_PROMO_CODES` を逆参照して3ノードの循環 Import を形成し、同時に `order_app/infra/_internal.py`（`/tmp/order_spool/fees.dat` 参照）へ直接結合しています。



アーキテクチャ層	LOC	対象パスと境界健全性メモ
API / Presentation	38	handlers.py, schemas.py (FastAPI+Flask同居・起動失敗)
Application Svc	60	services/processor.py (複雑度44・CWEシンク・Global辞書)
Domain Layer	18	domain/pricing.py (handlers逆参照で循環依存・float演算)
Infra & Scripts	124	gateway.py, _internal.py, legacy_sync.sh (/tmp・boto3未宣言)
Shared Core	15	constants.py, __init__.py (Final型完備・唯一の健全層)

図2-1: リポジトリ物理ディレクトリ構造 (TreeNode) とレイヤ別責務・規模一覧 (Table)

レイヤ間循環依存グラフとリスクホットスポット重畳

以下のコンポーネント依存図では、`module_hotspots` の実測リスクスコアを `HeatmapOverlay` として重畳しています。`Dockerfile`・`order_app/services/processor.py`・`order_app/schemas.py`・`order_app/api/handlers.py`（いずれも `risk_score=1.00`）を筆頭に、`tests/test_orders.py`（`0.90`）、`requirements.txt`（`0.84`）、

`order_app/infra/gateway.py`（`0.72`）、`order_app/domain/pricing.py`（`0.50`）と、ほぼ全層が高リスク帯（赤〜橙）に染まっています。

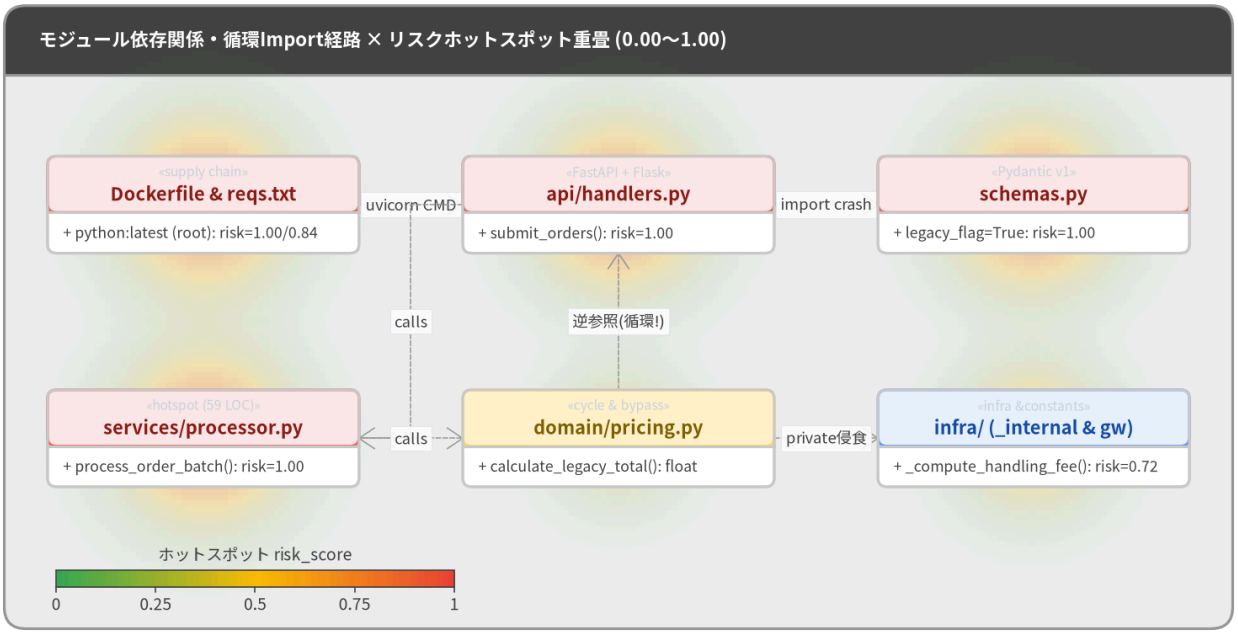


図2-2: レイヤ間循環依存・境界違反グラフとモジュール別リスクホットスポット (HeatmapOverlay)

実行フローと状態汚染・危険シンク経路

注文バッチ送信（`POST /orders`）から価格算出・インメモリキャッシュ記録に至る実行経路と、GitHub Actions `pull_request_target` から起動される `scripts/legacy_sync.sh` の流れは以下の5段階に整理されます。

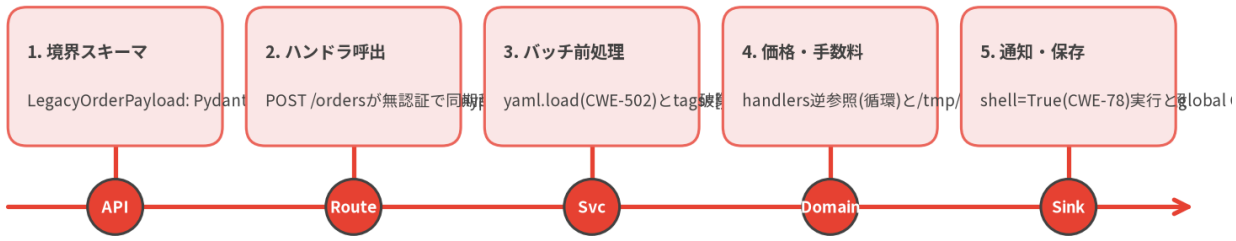


図2-3: 注文バッチ処理リクエスト経路に潜む状態汚染・循環参照・危険シンクの5段階フロー

全ソースモジュール・スクリプト一覧

モジュール相対パス	LOC	公開クラス / 型	公開関数・主要シンボル / 健全性診断
`order_app/services/processor.py`	59	—	`process_order_batch`, `ORDER_STATE_CACHE`, `WEBHOOK_SIGNING_SECRET`（複雑度44・CWE-502/78）
`order_app/api/handlers.py`	25	—	`app`, `legacy_flask`, `init_cache`, `submit_orders`, `legacy_health`, `ACTIVE_PROMO_CODES`
`order_app/domain/pricing.py`	17	—	`calculate_legacy_total`（`api.handlers` 循環参照・`infra._internal` 直参照・Docstring不一致）
`order_app/constants.py`	14	—	`is_supported_currency`, `SUPPORTED_CURRENCIES`, `DEFAULT_TAX_RATE`（`Final` 型・テスト100%）
`order_app/schemas.py`	12	`LegacyOrderPayload`, `Config`	`check_customer`（Pydantic v1 `@validator`・未注釈 `legacy_flag` で起動クラッシュ）
`order_app/infra/gateway.py`	11	—	`notify_partner_webhook`（`from math import *`・未宣言 `boto3`・`async` 内同期 `sleep` / `post`）
`order_app/infra/_internal.py`	6	—	`_compute_handling_fee`（`/tmp/order_spool/fees.dat` 存在有無で手数料分岐）
`order_app/**/__init__.py` (計5件)	5	—	ルートおよび `api` / `services` / `domain` / `infra` 各パッケージ初期化（各1 LOC）
`scripts/legacy_sync.sh`	106	—	`check_stage`, `rotate_archives`, `emit_metrics` 等（`/var/tmp` 操作・未使用関数残存）

3. 推奨する変更対応

検出された28件の指摘（`CRITICAL` 8件・`HIGH` 14件・`MEDIUM` 5件・`LOW` 1件）および14件の `FAIL` ルールを安全かつ段階的に解消するため、施策を **P0（即時緊急対応：7件）→ P1（構造・再現性・テスト基盤確立：5件）→ P2（保守性リファクタリング・ドキュメント整備：2件）** の計14アクションへ整理しました。まずは **Phase 1 (P0)** で秘密情報・特権CI・CWE危険シンク・コンテナ基盤CVE・起動時クラッシュを解消し、ステージ済 `.pni/pentests/` DAST ゲートおよび回帰テスト網を通過させた上で **Phase 2 (P1)** の循環依存解消へ進む手順を推奨します。

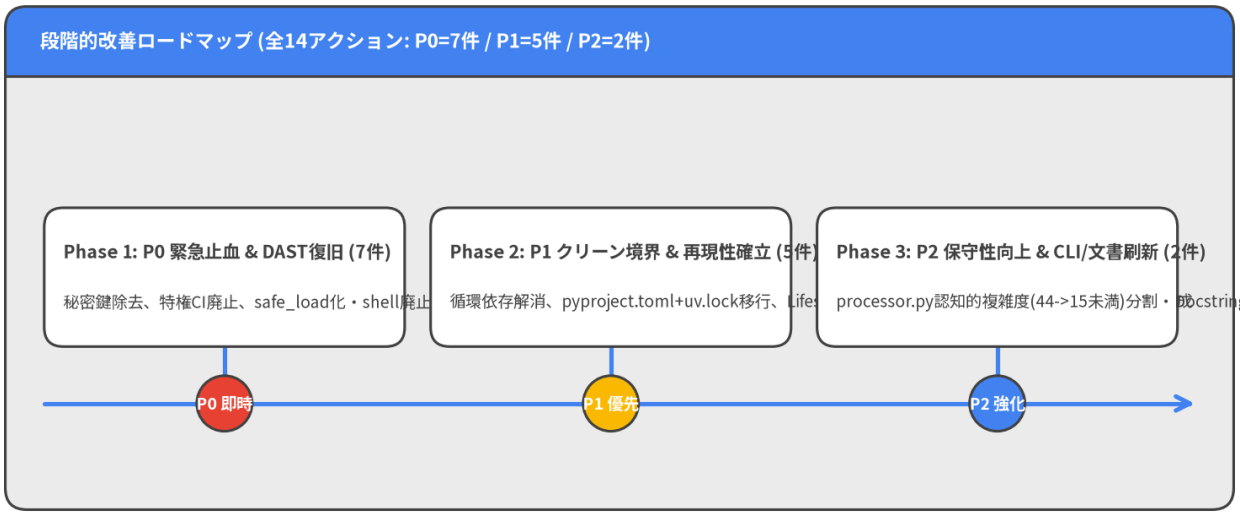


図3-1: 優先度順の3段階是正ロードマップ (P0 緊急止血・DAST起動復旧 -> P1 境界・ロック・網羅率確立 -> P2 複雑度低減・文書化)

最優先（P0〜P1）変更対応マトリクス

以下の図表は、即時着手が求められる P0 アクション群と中核アーキテクチャ改善 P1 代表施策の対象ファイル・期待効果・関連ルール対応を示しています。

優先度	施策	対象ファイル	期待効果	関連ルール
P0	秘密鍵外部化＆ダミー鍵削除	processor.py, config/*.json	ソース・イメージからの秘密情報完全除去	common/security
P0	特権CI廃止＆Action SHA固定	deploy.yml, Dockerfile	PR権限昇格防止・非rootコンテナ実行	common/supply_chain
P0	CWE-502/78解消＆API認証追加	processor.py, handlers.py	RCE/OSコマンド注入根絶・境界防御確立	security_vulnerability
P0	HTTP起動復旧＆DASTゲート通過	schemas.py, handlers.py, reqs	server_booted復旧・CWE-306/20プローブ合格	CWE-306, CWE-20
P0	python:3.12-slim多段化＆CVE解消	Dockerfile, requirements.txt	Python HIGH 4件・OS CRIT 24件の解消	CVE-2025-47273 他
P1	循環依存解消＆uv.lock・テスト網羅	pricing.py, tests/test_orders.py	単方向DAG回復・行カバレッジ85%以上	architecture, testing

図3-2: 最優先改善施策（優先度・施策・対象ファイル・期待効果・関連ルール）

全14件の改善アクション一覧（完全トレーサビリティ表）

#	優先度	施策タイトルと具体実装内容	対象ファイル	期待効果
1	P0	**ハードコード秘密鍵の外部化とコミット済鍵ファイルの削除**: `WEBHOOK_SIGNING_SECRET` を環境変数設定へ移行し、`config/service_account_dummy.json` を履歴・ビルド文脈から除外する。	`order_app/services/processor.py`, `config/service_account_dummy.json`	ソース・コンテナ層からの秘密情報漏洩リスクを根絶。
2	P0	**GitHub Actions 最小権限化とコンテナビルド衛生の確立**: `pull_request_target` を `pull_request` に変更し、`permissions: contents: read` と 40文字コミットSHA固定、`.dockerignore` を導入する。	`.github/workflows/deploy.yml`, `Dockerfile`, `config/service_account_dummy.json`	未信頼 PR による任意コード実行・権限昇格を遮断。
3	P0	**CWE-502/CWE-78 危険シンク除去・可変状態修正・API認証強制**: `yaml.safe_load()` への置換、`subprocess.run(..., shell=True)` 廃止、`tags=[]` と `ORDER_STATE_CACHE` 排除、`POST /orders` 認証追加を行う。	`order_app/services/processor.py`, `order_app/api/handlers.py`, `order_app/schemas.py`	RCE/コマンド注入シンクとリクエスト間タグ汚染バグを解消。
4	P0	**コンテナ HTTP 起動復旧と `.pni/pentests/` DAST ゲート通過**: `legacy_flag: bool = True` と型付き `OrderItem` DTO 導入、`uvicorn` 固定、`POST /orders` 認可・セキュリティヘッダ付与後、`web_pentest` 全プローブを通過させる。	`order_app/schemas.py`, `order_app/api/handlers.py`, `requirements.txt`	`server_booted=true` を回復し、`CWE-306`（`401` / `403`）・`CWE-20`（`422`）・HTTP ヘッダ防御を動的に保証。
5	P0	**脆弱 Python パッケージ更新と `python:latest` root 基盤の刷新**: `python:3.12-slim` マルチステージ＋非root `USER` ＋ `HEALTHCHECK` へ移行し、`urllib3>=2.8.0` 固定および不要 `flake8` / `black` / `setuptools` 除外を行う。	`Dockerfile`, `requirements.txt`, `setup.py`	Python HIGH CVE 4件と Debian `-dev` 由来の 3,554 OS CVE の大半を一掃。
6	P0	**アサーション付きミラー単体・統合テストスイート構築**: `pytest` / `pytest-cov` / `httpx` を dev 依存へ追加し、API・バッチ処理・価格計算の正常／異常系テストを整備する。	`tests/test_orders.py`, `order_app/api/handlers.py`, `order_app/services/processor.py`, `order_app/domain/pricing.py`	安全なリファクタリングのための回帰検知網（85%以上）を確立。
7	P0	**起動時 `PydanticUserError` 修正と `uvicorn` 依存の明示**: `schemas.py` の `legacy_flag: bool = True` 型注釈付与と Pydantic v2 移行、`uvicorn` 宣言、FastAPI `GET /healthz` 実装を行う。	`order_app/schemas.py`, `order_app/api/handlers.py`, `requirements.txt`, `Dockerfile`	ASGI アプリの正常インポート・起動スモーク（RV-04）を復旧。

#	優先度	施策タイトルと具体実装内容	対象ファイル	期待効果
8	P1	**循環依存の解消・クリーンレイヤ境界と型注釈 100% 化**: `ACTIVE_PROMO_CODES` を `constants.py` へ移設し、`pricing.py` の `handlers` / `_internal` 参照と Flask 同居・未宣言 `boto3` を撤去、全関数に型を付与する。	`order_app/domain/pricing.py`, `order_app/api/handlers.py`, `order_app/infra/_internal.py`, `order_app/infra/gateway.py`, `order_app/constants.py`	単方向 DAG アーキテクチャ回復と静的型検査ゼロエラーを達成。
9	P1	**FastAPI Lifespan・`/healthz`・型付き設定・非同期 Timeout 実装**: `pydantic-settings` による設定一元化、`@asynccontextmanager lifespan`、`httpx.AsyncClient(timeout=5.0)`、構造化 `logging` を導入する。	`order_app/api/handlers.py`, `order_app/infra/gateway.py`, `order_app/services/processor.py`, `order_app/infra/_internal.py`	イベントループブロッキング防止とマルチレプリカ無状態運用を実現。
10	P1	**OpenSSF ガバナンス文書と Dependabot 自動更新の整備**: `.gitignore`・`LICENSE`・`SECURITY.md`・`.github/dependabot.yml` を追加し、PR 自動テストゲートを構成する。	`.github/workflows/deploy.yml`, `requirements.txt`	サプライチェーン統制と脆弱性検知の継続的自動化を実現。
11	P1	**PEP 621 `pyproject.toml` + `uv.lock` への完全移行**: レガシー `setup.py` と未ピン `requirements.txt` を廃止し、実行時依存と `[dependency-groups].dev` を厳格分離する。	`requirements.txt`, `setup.py`	100% 決定論的な依存解決と不要パッケージ混入防止を実現。
12	P1	**カバレッジ 0.0% モジュール群の解消（6.6% → 85%以上）**: 未実行だった主要6モジュール（計99文）に対するオフライン単体・HTTP テストを追加し、`--cov-fail-under=80` を強制する。	`tests/test_orders.py`, `order_app/services/processor.py`, `order_app/domain/pricing.py`, `order_app/infra/gateway.py`	全ビジネス分岐・エラー経路の実行保証を確立。
13	P2	**バッチ処理関数の複雑度分割と正確な Docstring 整備**: `process_order_batch`（認知的複雑度 `44`）を責務別ヘルパーへ分割し、全公開シンボルの Google 形式 Docstring を同期する。	`order_app/services/processor.py`, `order_app/domain/pricing.py`, `order_app/api/handlers.py`	認知的複雑度を 65% 以上削減し、保守性と API 仕様明確性を向上。
14	P2	**レガシー Bash 同期スクリプトの Python CLI 化と README 拡充**: `scripts/legacy_sync.sh`（106 LOC）を型付き Python CLI へ置換し、Standard Readme 準拠の運用・開発手順書を整備する。	`README.md`, `scripts/legacy_sync.sh`	シェル解析バグを排除し、開発者オンボーディング時間を短縮。

4. 検査指標・ルール適合マトリクスと指摘一覧

本ワークフロー（`py_api_legacy_messy` / `wf-fe76f694`）では、8つの AI 解釈ステップと5つの決定論的検査（`project_inspect`・`ast_analyze`・`run_check`・`security_scan`・`web_pentest`）を通じて全15エンジニアリングルールを検証しました。結果は `FAIL` **14件**・`WARN` **1件**・`PASS` **0件（適合率** `0.0%` **）**、検出指摘は計 **28件（** `CRITICAL` **8件**・`HIGH` **14件**・`MEDIUM` **5件**・`LOW` **1件）** です。全8監査ステップのスコアがいずれも Grade D 帯（`10〜22` 点）に沈んでいます。

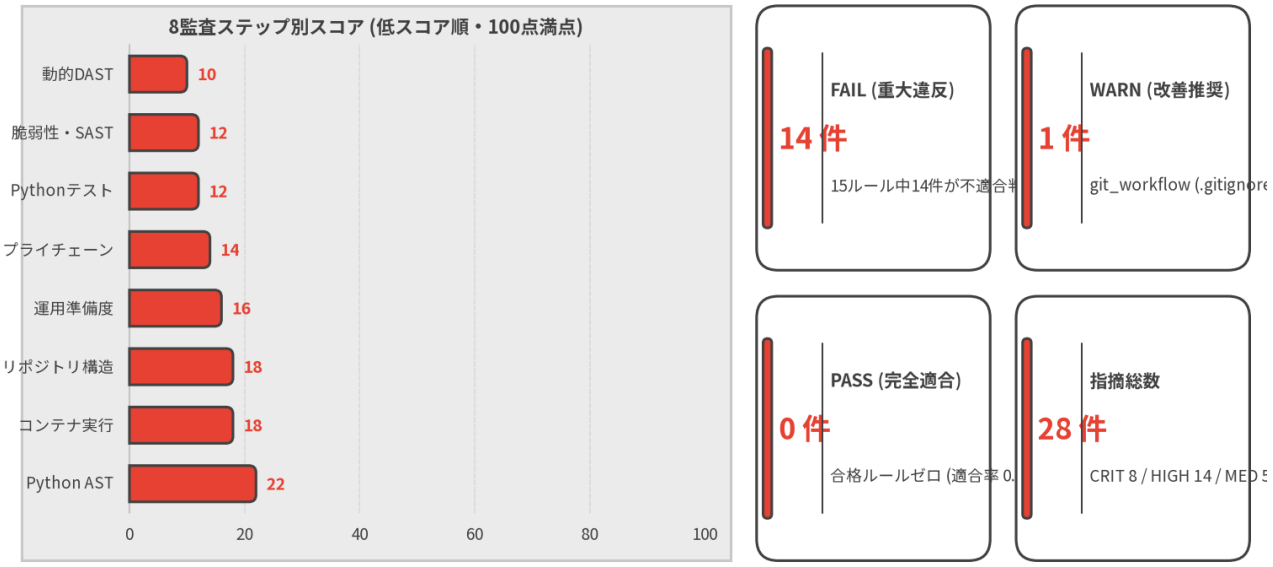


図4-1: 8監査ステップ別スコア比較（低スコア順）とルール適合・指摘重大度KPI

ルール適合マトリクス（`FAIL -> WARN -> PASS`）と重大指摘サマリー

以下の図4-2では評価された主要エンジニアリングルールの不適合証跡を、図4-3では最優先で是正すべき `CRITICAL` 指摘8件を一覧化しています。

判定	ルールID	決定論的検出証跡 (Evidence)	PASS化への必須要件
FAIL	common/security_vulnerability	CVE 3,561件・CWE-502/78・DAST起動不能(error)	slim化・safe_load・起動復旧&DAST合格
FAIL	common/runtime_verification	行網羅率6.6%・分岐0.0%・起動PydanticUserError	schemas修正・行85%以上のテスト網羅
FAIL	common/supply_chain	ロック無・pull_request_target・@main・root実行	uv.lock・pull_request最小権限・非root化
FAIL	common/architecture	循環依存3経路・domain->handlers/infra._internal	ACTIVE_PROMO_CODES移設・単方向DAG化
FAIL	common/maintainability	認知的複雑度44・8引数・tags=[]・global辞書	DTO引数化・状態注入・関数分割
FAIL	python/typing	型網羅44.4%・未注釈9箇所・無制限type:ignore	全公開関数型注釈・Pydantic v2厳格化
FAIL	python/testing	12ソースに対しassert無し2行テスト1件のみ	ミラー構成pytestスイートとHTTP検証
WARN	common/git_workflow	.gitignore不在・PR上でdeployスクリプト実行	.gitignore追加とPR自動検証ゲート分離

図4-2: 主要ルール適合マトリクス抜粋 (FAIL -> WARN -> PASS 順)

重大度	対象箇所	検出証跡 (Evidence)	推奨される是正措置
CRIT	processor.py:8	WEBHOOK_SIGNING_SECRET ハードコード	コードから即時削除・環境変数/Secret管理へ移行
CRIT	Dockerfile:1	python:latest (root)・OS CRIT 24 / HIGH 436	python:3.12-slim固定・非root USER・多段ビルド化
CRIT	schemas.py:9 (DAST)	web_pentest server_booted=false (3.52s)	legacy_flag:bool付与・起動復旧しDASTゲート実行
CRIT	processor.py:30,63	yaml.load (CWE-502) & shell=True (CWE-78)	yaml.safe_load採用・shell=True呼び出し完全廃止
CRIT	deploy.yml:4,11	pull_request_target + checkout@main 未固定	pull_request化・contents:read・40文字SHA固定
CRIT	tests/test_orders.py:4	実測行カバレッジ6.6% (7/106)・分岐0.0%	主要6モジュールの単体/TestClient検証を追加
CRIT	tests/test_orders.py:4	test_currency_smoke に assert 文がゼロ	明示的 assert 追加と異常系・境界値テスト整備
CRIT	schemas.py:9 (Smoke)	未注釈 legacy_flag=True でASGI起動失敗	bool型注釈付与とPydantic v2モデルへ移行

図4-3: 最重大 (CRITICAL) 指摘8件の箇所・証跡・推奨修正サマリー

全15ルール適合マトリクス一覧（`FAIL -> WARN -> PASS`

順)

判定	ルールID	主検出ステップ	決定論的証跡と判定サマリー
FAIL	`common/architecture`	`python_ast_audit`	循環依存 `3件`（`handlers -> processor` `pricing -> handlers`）、境界違反 `2件`（ `-> handlers`, `pricing -> infra._interr` FastAPI/Flask 同居。
FAIL	`common/documentation`	`repo_structure_audit`	`README.md` は2行（見出し1個）のみ。セ プ・環境変数・API仕様・テスト手順が欠 106行 Bash の説明も皆無。
FAIL	`common/maintainability`	`python_ast_audit`	`process_order_batch` が58行・16分岐・ 雑度 `44`・引数 `8個`（上限6超過）、`tags` `ORDER_STATE_CACHE`、裸の `except:` を伴
FAIL	`common/production_readiness`	`service_readiness_audit`	`requests.post` タイムアウト欠落・`asyn` 阻害、FastAPI `/healthz` 不在（未マウン に定義）、`uvicorn` 未宣言、起動クラッシ
FAIL	`common/runtime_verification`	`container_runtime_audit`	実測行カバレッジ `6.6%`（`7/106` 文）・ケ `0.0%`、ASGI 起動スモークが `schemas.py` `PydanticUserError`（`1.92s`, exit=1）で
FAIL	`common/security`	`repo_structure_audit`	`processor.py:8` のハードコード署名鍵、 `config/service_account_dummy.json` 同 `yaml.load`、`shell=True`、`Any` 境界入力
FAIL	`common/security_vulnerability`	`security_vulnerability_audit` / `web_penetration_audit`	Python CVE `7件`（HIGH 4件）、Debian O `3,554件`（CRIT 24 / HIGH 436）、IaC 不備 CWE-502/78、無認証 API、動的 `web_per` 失敗（`server_booted=false`）。
FAIL	`common/supply_chain`	`repo_structure_audit`	ロックファイル `0件`・未ピン依存 `8/8`、 `pull_request_target` + `checkout@main` `permissions` 欠落、root `:latest` コンテ `LICENSE` / `SECURITY.md` 不在。
FAIL	`common/twelve_factor`	`service_readiness_audit`	散在 `os.getenv`、インメモリ `ORDER_STATE_CACHE`、`/tmp/order_spool` 依存、`print()` 3箇所、非推奨 `@app.on_event("startup")`。
FAIL	`python/code_quality`	`python_ast_audit`	Docstring 網羅率 `40.9%`（欠落12件、`TO` `pricing.py:8` 引数名不一致1件）、裸の と `except Exception: pass`。
FAIL	`python/dependencies`	`repo_structure_audit`	`pyproject.toml` / `uv.lock` 不在、実行時 `flake8` / `black` / `urllib3` 混入、`uvicorn` （Dockerfile）と `boto3`（`gateway.py`） 言。
FAIL	`python/modern_stack`	`python_ast_audit`	Import衛生 `75.0%`（`isort` 違反4件、`fr` `import *`、関数内 `import json`）、Pyda

判定	ルールID	主検出ステップ	決定論的証跡と判定サマリー
			`@validator` / `Config` 、 `os.path` 利用。
FAIL	`python/testing`	`python_test_audit`	12ソースに対しテストファイル1件・関数 み、かつ `assert` 文が `0個` （アサーショ テスト）。
FAIL	`python/typing`	`python_ast_audit`	型安全性カバレッジ `44.4%` （未注釈7箇所 け `dict` 1箇所、 `schemas.py:9` 無制限 `# ignore` 1箇所、 `py.typed` 無し）。
WARN	`common/git_workflow`	`repo_structure_audit`	`.gitignore` 不在によりビルド生成物や秘 リスクがあり、PR トリガーでテストでは スクリプトを実行。

全28件の指摘一覧（`CRITICAL -> HIGH -> MEDIUM -> LOW`

順)

#	重大度	ルールID	箇所	検出された証跡と推奨修
1	CRITICAL	`common/security`	`order_app/services/processor.py:8`	`WEBHOOK_SIGNING_SECRE` `"sk_live_***REDACTED*` き。即時削除・ローテー 境変数設定へ移行。
2	CRITICAL	`common/security_vulnerability`	`Dockerfile:1`	`FROM python:latest` (D に OS CVE 3,554件（CR 436）および `DS-0002` r `python:3.12-slim` 非ro
3	CRITICAL	`common/security_vulnerability`	`order_app/schemas.py:9`	`web_pentest` （`http://127.0.0.1:54` `3.52s`）が `status="er` `server_booted=false` `field-missing-annotat` 失敗し全 DAST プローフ 注釈付与と起動復旧。
4	CRITICAL	`common/security_vulnerability`	`order_app/services/processor.py:30`	`yaml.load(override_ya` `502`）と L63 `subprocess` `shell=True)`（`CWE-78`）。 `yaml.safe_load` 化と `s` 廃止。
5	CRITICAL	`common/supply_chain`	`.github/workflows/deploy.yml:4`	`pull_request_target` + `actions/checkout@main` `permissions` 未設定で `legacy_sync.sh` を実行 `pull_request` 化・SHA
6	CRITICAL	`common/runtime_verification`	`tests/test_orders.py:4`	`pytest-cov` 実測行カバ `6.6%`（`7/106`）・分岐 `0` モジュール全て `0.0%`）。 HTTP テストで 80%以上
7	CRITICAL	`python/testing`	`tests/test_orders.py:4`	`test_currency_smoke()` が皆無。明示的アサーシ ー構成の pytest スイート
8	CRITICAL	`common/runtime_verification`	`order_app/schemas.py:9`	未注釈 `legacy_flag = ` Pydantic v2 下で `PydanticUserError` が 起動スモーク失敗。型注 Pydantic v2 移行。
9	HIGH	`common/architecture`	`order_app/domain/pricing.py:3`	`order_app.api.handler` による循環依存と `infra.` 直接結合。定数を `const` 移し Decimal 演算化。

#	重大度	ルールID	箇所	検出された証跡と推奨修
10	HIGH	`common/production_readiness`	`order_app/api/handlers.py:31`	`/legacy/health` が未マ `Flask` 側に定義され Fa スプローブが無い。Fast `/healthz` へ統一。
11	HIGH	`common/twelve_factor`	`order_app/services/processor.py:21`	`global ORDER_STATE_CA []`・散在 `os.getenv`・ ル参照。`pydantic-sett` Repository 注入へ移行。
12	HIGH	`common/maintainability`	`order_app/services/processor.py:11`	8引数・認知的複雑度 `4 []`・裸の `except:`（L3 り潰し（L64）。DTO化と 分割を実施。
13	HIGH	`common/production_readiness`	`order_app/infra/gateway.py:11`	`async def` 内で `time.s` `timeout=` 無しの `reque` 実行。 `httpx.AsyncClient(tim` へ置換。
14	HIGH	`common/modern_stack`（`python/modern_stack`）	`order_app/infra/gateway.py:7`	`from math import *` と `boto3` インポート（ク `ImportError`）。不要イ 除と非同期 HTTP 化。
15	HIGH	`common/production_readiness`	`Dockerfile:6`	`CMD ["uvicorn", ...]` `requirements.txt` に `u` 宣言。実行時依存に明記 `HEALTHCHECK` を追加。
16	HIGH	`common/supply_chain`	`Dockerfile:1`	`:latest` タグ・root 実 行・`.dockerignore` 欠 `config/` 鍵ファイルが 入。非root slim 化と除外
17	HIGH	`common/supply_chain`	`requirements.txt:1`	全8依存が未ピン・ロッ `LICENSE` / `SECURITY.md` 欠落。`uv.lock` と Oper ンス文書を整備。
18	HIGH	`python/dependencies`	`requirements.txt:1`	`setup.py` + `requiremer` `flake8` / `black` が混在 `uvicorn` / `boto3` が欠落 `pyproject.toml` へ移行
19	HIGH	`common/security_vulnerability`	`order_app/api/handlers.py:17`	ステージ済 `.pni/pentes`（`test_app_probes.py` 認証 `POST /orders`・`C` 付け `items` 500 検証） で `0/0` 未実行。修正後 ート化。
20	HIGH	`common/security_vulnerability`	`order_app/api/handlers.py:18`	`POST /orders` が認証無 `response_model` 無し・

#	重大度	ルールID	箇所	検出された証跡と推奨修
				`Any]` 入力で `customer_` 認証依存と厳格 `OrderI` 実装。
21	HIGH	`common/security_vulnerability`	`requirements.txt:6`	Python 依存 CVE 7件（H `setuptools@70.3.0`, `urllib3@2.7.0` ×2, `msgpack@1.1.2`）。不要 `2.8.0+` 固定。
22	HIGH	`common/supply_chain`	`Dockerfile:1`	`COPY . .` と root 実行に 拡大。マルチステージ `u` `frozen --no-dev` と最小 を採用。
23	MEDIUM	`common/supply_chain`	`scripts/legacy_sync.sh:1`	106行の Bash スクリプ `u` / `pipefail` 無しで `/v` 作。型付き Python CLI へ刷新。
24	MEDIUM	`common/twelve_factor`	`order_app/services/processor.py:25`	`print()` 出力3箇所およ 記録握り潰し。標準 `lo` る構造化ログとエラー記
25	MEDIUM	`common/documentation`	`README.md:1`	2行のみのスタブ文書。 チャ・環境構築・設定変 例・テスト実行手順を網 README へ拡充。
26	MEDIUM	`common/security`	`config/service_account_dummy.json:1`	サービスアカウント形式 リポジトリ管理下に存在 から除外し `.gitignore` / `.dockerig` 断。
27	MEDIUM	`python/typing`	`order_app/schemas.py:5`	Pydantic v1 `@validator` / `Config.or` 変 `metadata: dict = {}` `ignore` 。Pydantic v2 `C` へ更新。
28	LOW	`python/code_quality`	`order_app/domain/pricing.py:8`	Docstring が存在しない `raw_items` / `discount_p` （他12箇所で Docstring グネチャへ同期。

5. 詳細分析：コード品質・テスト・動的検証

Python AST 静的解析（全12ファイル・`149 LOC`・2クラス・9関数）では、**Import衛生** `75.0%`・**型安全性** `44.4%`・**Docstring網羅率** `40.9%` に留まり、`order_app/services/processor.py` の `process_order_batch`（58行・16分岐・**認知的複雑度** `44`・8引数・可変デフォルト `tags=[]`・`global ORDER_STATE_CACHE`）に深刻な保守性負債が集中しています。さらに Docker 隔離環境での動的検証（`run_check`）では、`Dockerfile` ビルド（`1.00s`）とテストハーネス構築（`0.96s`）および1件のオフライン `pytest`（`2.13s`）こそ終了コード `0` で通過したものの、**実測カバレッジは行** `6.6%`（`7/106` **文）・分岐** `0.0%` に過ぎず、続く **ASGI 起動スモーク検証（**`smoke_check`**,** `1.92s`**）および隔離ループバック HTTP 動的検証（**`web_pentest`**,** `3.52s`**）の双方が** `order_app/schemas.py:9` **の** `PydanticUserError` **で即座にクラッシュ** しました。

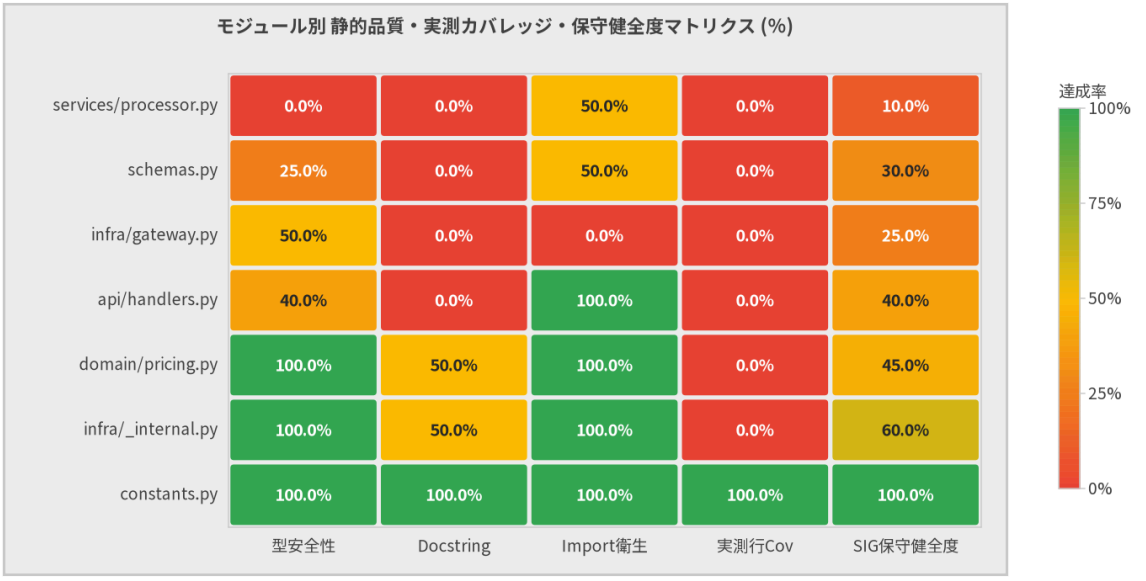
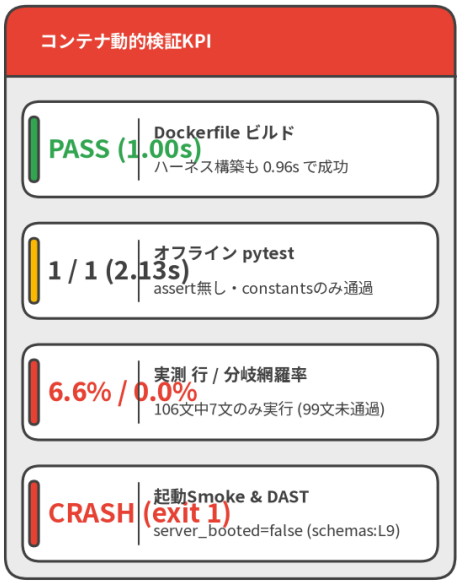


図5-1: 主要7モジュール × 5品質指標の健全性ヒートマップ（静的AST指標 vs 実測行・分岐カバレッジ）

コンテナ動的検証ステージとカバレッジ 0.0% モジュール内訳

`tests/test_orders.py` が `order_app.constants` しかインポートしていないため、`pytest` 実行時には `handlers.py` や `schemas.py` が一度も読み込まれず、`smoke_check`（`importlib.import_module("order_app.api.handlers")`）および `web_pentest`（`uvicorn` コンテナ起動 → `http://127.0.0.1:54305` ポーリング）を実行した瞬間に初めて Pydantic v2 の未注釈フィールド検査例外（`legacy_flag = True`）が顕在化しました。



モジュール (lowest_covered)	行 / 分岐	実行文数・未実行理由
services/processor.py	0.0% / 0%	0 / 45 文 (中核バッチ処理が完全未テスト)
api/handlers.py	0.0% / —	0 / 17 文 (import時にschemasで例外停止)
schemas.py	0.0% / —	0 / 12 文 (legacy_flag未注釈クラッシュ)
infra/gateway.py	0.0% / —	0 / 10 文 (未宣言boto3でImportError潜在)
domain/pricing.py	0.0% / 0%	0 / 9 文 (割引・手数料・税計算が未検証)
infra/_internal.py	0.0% / 0%	0 / 6 文 (/tmp/order_spool分岐が未検証)
constants.py	100% / —	7 / 7 文 (唯一テストから呼出、assert無し)

図5-2: コンテナ動的検証 (run_check & web_pentest) KPI（左）とステージ・最低カバレッジ内訳（右）

動的検証・`pytest-cov` 実測結果と起動障害の詳細診断

検証項目 / モジュール	ステータス / 計測値	詳細証跡 (`RuntimeFacts` / `PentestFacts`) と技術的原因
`project_dockerfile_build`	`passed` (`1.00s`)	`Dockerfile` （ `FROM python:latest` ）のビルド自体は成功（ただし `uvicorn` は `fastapi` 推移的依存頼み）。
`harness_build`	`passed` (`0.96s`)	`python/pip.dockerfile` （Python 3.12）テストハーネスのビルド成功。
`test_execution` (`--network=none`)	`passed` (`1/1` , `2.13s`)	`tests/test_orders.py::test_currency_smoke` 1件のみ実行。 `assert` 無しのため戻り値を一切検証せず終了。
`smoke_check` (`web_api`)	`failed` (`exit=1` , `1.92s`)	`order_app.api.handlers` インポート時に `order_app/schemas.py:9` で `pydantic.errors.PydanticUserError: A non-annotated attribute was detected: legacy_flag = True` が送出され起動不能。
`web_pentest` (`http://127.0.0.1:54305`)	`error` (`server_booted=false` , `3.52s`)	実コンテナ HTTP 起動時も同一の `For further information visit https://errors.pydantic.dev/2.14/u/model-field-missing-annotation` で即座に終了し、ライブ DAST プローブ実行数 `0/0` 。
`order_app/services/processor.py`	**行** `0.0%` **/ 分岐** `0.0%`	`0 / 45` 文（未通過行: `1-3, 5, 7-8, 11, 22, 24-25...` ）。16分岐すべて未通電。
`order_app/api/handlers.py`	**行** `0.0%`	`0 / 17` 文（未通過行: `1-2, 4-5, 7-9, 12-14...` ）。 `POST /orders` ・起動イベントともに未検証。
`order_app/schemas.py`	**行** `0.0%`	`0 / 12` 文（未通過行: `1-2, 5-9, 11-12, 14...` ）。Pydantic v2 非互換コードが潜在。
`order_app/infra/gateway.py`	**行** `0.0%`	`0 / 10` 文（未通過行: `1-4, 7, 9-13` ）。インポートすると `ModuleNotFoundError: No module named 'boto3'` が発生。
`order_app/domain/pricing.py`	**行** `0.0%` **/ 分岐** `0.0%`	`0 / 9` 文（未通過行: `3-5, 8, 18-22` ）。クーポン割引や税率丸め計算が未検証。
`order_app/infra/_internal.py`	**行** `0.0%` **/ 分岐** `0.0%`	`0 / 6` 文（未通過行: `1, 4-8` ）。 `/tmp/order_spool/fees.dat` 分岐が未検証。
`order_app/constants.py` + `__init__.py`	**行** `100.0%`	`7 / 7` 文（ `missing_lines=[]` ）。唯一の実行済モジュール。

6. 詳細分析：セキュリティ・脆弱性・動的DAST・API境界

Trivy（`0.69.3`）によるファイルシステムおよび解決済コンテナイメージ検査（`security_scan`）、Python AST SAST 解析、および隔離ループバック HTTP 動的ペネトレーション検証（`web_pentest`）の結果、本リポジトリは **サプライチェーン・コンテナ基盤・アプリケーションコード・CI/CD・HTTP 境界・動的可用性の全6層に重大な脆弱性** を抱えています。具体的には、Python パッケージ層で **計7件（HIGH 4件 / MEDIUM 2件 / LOW 1件）**、フルサイズの `FROM python:latest`（Debian 13.7）OS パッケージ層で **計3,554件（CRITICAL 24件 / HIGH 436件 / MEDIUM 1,308件 / LOW 1,654件）** の CVE が検出されたほか、Dockerfile 設定不備3件（`DS-0002` root 実行等）、AST 危険シンク2件（`CWE-502`・`CWE-78`）、ハードコード秘密鍵、コミット済ダミー認証情報、無認証 API、さらに `web_pentest` **実行時のコンテナ起動クラッシュ（`status="error"`, `server_booted=false`）** が確認されました。

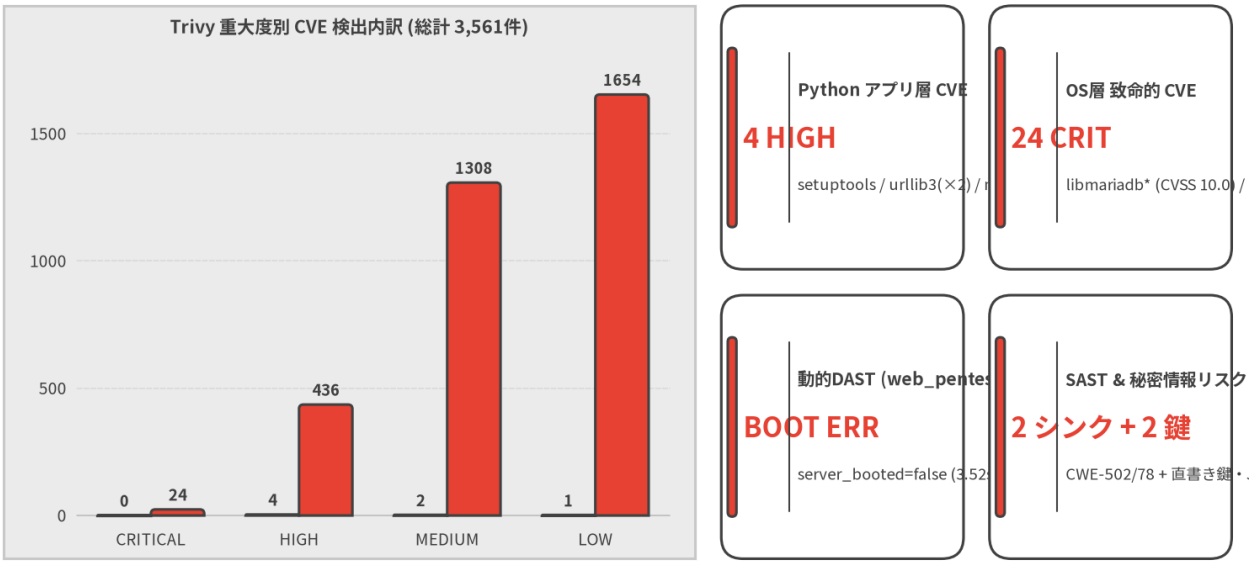


図6-1: Python依存 vs Debian 13.7 OS層の重大度別CVE件数（左）とコンテナ・DAST・サプライチェーンリスクKPI（右）

AST 危険コードシンク（CWE）・動的 DAST・無認証 API・優先パッチ CVE 一覧

コード内部では `order_app/services/processor.py` が安全なローダ指定のない `yaml.load(override_yaml)`（**CWE-502**：任意 Python オブジェクト生成・RCE）と `subprocess.run(f"echo batch {batch_id} done", shell=True)`（**CWE-78**：OS コマンドインジェクション）を含み、HTTP 境界では `POST /orders` が認証ヒント無し（`has_auth_hint=false`）で公開されています。

区分 / 重大度	識別子 (CWE/CVE)	検出箇所 / パッケージ	攻撃リスクと具体的な緩和・修正バージョン
DAST [CRIT]	SEC-06 (Boot失敗)	schemas.py:9 (54305/tcp)	server_booted=false (3.52s) -> legacy_flag型注釈付与・起動復旧
DAST [HIGH]	CWE-306 / CWE-20	POST /orders (test_app_probes)	無認証200 & 未型付けitems 500 -> 認可強制・OrderItem厳格DTO化
SAST [HIGH]	CWE-502	services/processor.py:30	yaml.load(override_yaml) 任意コード実行 -> yaml.safe_load() へ置換
SAST [HIGH]	CWE-78	services/processor.py:63	subprocess.run(..., shell=True) 注入 -> shell廃止・構造化ログ化
API [MED]	SEC-05 (未接続)	GET /legacy/health (L31)	未マウントFlask定義 -> FastAPI GET /healthz へ移設・公開範囲明示
PyPkg [HIGH]	CVE-2025-47273	setuptools@70.3.0 (CVSS 8.8)	PackageIndex パストラバーサル -> 78.1.1 更新または本番イメージ除外
OS/IaC [CRIT]	CVE-2026-85746 他	python:latest / DS-0002	MariaDB/OpenEXR等 CRIT 24件 & root実行 -> python:3.12-slim 非root化

図6-2: AST CWE危険シンク・動的DASTゲート・無認証API・優先修正CVEの統合セキュリティマトリクス

動的 HTTP ペネトレーション検証（`web_pentest`）とステージ済 DAST スイート

隔離ループバックコンテナ上でプロジェクト `Dockerfile` を起動して HTTP 攻撃耐性を実測する `web_pentest`（対象: `http://127.0.0.1:54305`、実行時間 `3.52s`）は、ASGI インポート直後に `order_app/schemas.py:9`（`legacy_flag = True # type: ignore`）でクラッシュし、`status="error"`・`server_booted=false`（`boot_failure_reason`: `"For further information visit https://errors.pydantic.dev/2.14/u/model-field-missing-`

`annotation"`）を記録しました。これによりサービスの動的可用性喪失が実証されるとともに、`.pni/pentests/examples/python/api_legacy_messy/` に生成・整備済みの4ファイル（計15テスト項目）は起動復旧後の必須 CI DAST ゲートとして待機状態（実行 `0/0`）となっています。

プローブファイル（`.pni/pentests/.../`）	対象エンドポイント	検証 CWE / カテゴリ	想定重大度	プローブ検証内容と起動復旧後の合格条件
`test_app_probes.py`	`POST /orders`	`CWE-306`（`missing_authentication`）	**HIGH**	認証ヘッダ無しの注文バッチ投入が `2xx` で受理されないこと（`401` / `403` 必須）。
`test_app_probes.py`	`POST /orders`	`CWE-20`（`improper_input_validation`）	**MEDIUM**	`items: List[Dict[str, Any]]` への欠損辞書投入で未処理 `500` を起こさず `422` で拒否すること。
`test_baseline_http.py`	`/` 〜 `/api/health`, `/.env`, `/.git/config` 他	`CWE-693`, `CWE-1021`, `CWE-200`, `CWE-942`, `CWE-538`	**CRIT〜LOW**	`X-Content-Type-Options: nosniff`、CORS 反射抑止、機密ファイル非公開、`TRACE` 無効化（計9項目）。
`test_target_surface.py`	`GET /legacy/health`	`CWE-79`, `CWE-89`, `CWE-209`, `CWE-22`	**CRIT〜MED**	クエリ反射 XSS・SQL エラー／スタックトレース漏洩・`../` パストラバーサル耐性（計4項目）。

主要 CVE・IaC 設定不備・秘密情報・API 境界の詳細対照

表

区分	重大度	識別子	対象（現在バージョン / 箇所）	修正先（`fixed_version` / 対策）	詳細と推奨アク
Python 依存	**HIGH**	`CVE-2025-47273`	`setuptools@70.3.0` (CVSS `8.8`)	`78.1.1`（または本番除外）	`PackageIndex` ラバーサル。マテージ `uv` ビルンタイムから除
Python 依存	**HIGH**	`CVE-2026-97687`, `CVE-2026-97689`	`urllib3@2.7.0` (CVSS `8.1`, `7.5`)	`2.8.0`	HTTPS プロキシ設定上書きおよンク解析 DoS。用宣言を削除し `>=2.8.0` に固定
Python 依存	**HIGH**	`GHSA-6v7p-g79w-8964`	`msgpack@1.1.2` (CVSS `7.5`)	`1.2.1`	エラー捕捉後の `Unpacker` 再利境界外読み取り `flake8` / `black` 番から分離し更
OS (Debian 13.7)	**CRITICAL**	`CVE-2026-85746`, `CVE-2026-92262`, `CVE-2026-49261`	`libmariadb3`, `libmariadb-dev` (`1:11.8.6-0+deb13u1`, CVSS `10.0` / `9.8`)	`python:3.12-slim` / distroless 移行	フルサイズ `python:latest` 梱する不要な D ライブラリ群の性。最小ベースジ化で除去。
OS (Debian 13.7)	**CRITICAL**	`CVE-2026-42217`, `CVE-2026-6653`, `CVE-2026-60002`	`libopenexr`, `libxml2`, `openssh-client` (CVSS `9.8` 〜 `9.4`)	最小ベースイメージ移行 + `apt-get upgrade`	画像・XML・SS ライアント開発ージをランタイ排除して攻撃面化。
IaC (`Dockerfile`)	**HIGH / MED / LOW**	`DS-0002`, `DS-0001`, `DS-0026`	`Dockerfile:1-6`	非root `USER`・ダイジェスト固定・`HEALTHCHECK`	UID 10001 等の実行ユーザ指定、`:latest` 廃止、`/healthz` チェック命令を
秘密情報 (`SEC-03`)	**CRITICAL / MED**	`OWASP-ASVS / S105`	`processor.py:8` (`sk_live_***REDACTED***`), `config/service_account_dummy.json`	環境変数注入 + Git/Docker 除外	ハードコード署クンとコミット JSON ファイル し、シークレッャンを CI 化。
CI/CD (`Supply`)	**CRITICAL**	`OpenSSF Token-Permissions`	`.github/workflows/deploy.yml:4,11`	`pull_request` + `contents: read` + SHA固定	特権 `pull_request_` と可変 `@main` 参廃止し、未信頼ード実行を防止